Reviewer Pack — Minimal Geometric Entanglement and Circuit Monotone Width Re…

Entry 4416fd13c45a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 08:31:35 UTC

Minimal Geometric Entanglement and Circuit Monotone Width Relationship

Entry ID: `4416fd13c45a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 08:31:35 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Geometric Entanglement) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every CNF φ, the minimal geometric entanglement of its associated state ρ is linearly correlated with its circuit monotone width w(ρ), such that mge(ρ) = Θ(w(ρ)) where mge(ρ) is the minimal geometric entanglement and w(ρ) is the circuit monotone width.

Rationale (proposer's reasoning):

> Geometric entanglement captures non-classical correlations in quantum systems, while circuit monotone width measures the minimum size of monotone Boolean circuits that can compute a given function. This conjecture suggests a potential connection between these two concepts, which could provide new insights into both complexity theory and quantum information.

Taxonomy category: `QIT_to_CircuitComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7d382990863b3cd2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between minimal geometric entanglement (mge) and circuit monotone width (w) across 30 random seeds exceeds 0.8, with no seed showing mge > 2 * w.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def truth_table_to_quantum_state(cnf):
        n = len(cnf[0])
        state = [0] * (1 << n)
        for assignment in itertools.product([-1, 1], repeat=n):
            if all(any(assignment[abs(lit) - 1] == l > 0 for l in clause) or any(assignment[abs(li
                state[tuple(assignment)] += 1
        return state

    def calculate_minimal_geometric_entanglement(state, n):
        # Placeholder function to compute minimal geometric entanglement
        # This is a dummy implementation and should be replaced with actual computation
        return sum(state) / (2 ** n)

    def calculate_circuit_monotone_width(cnf):
        # Placeholder function to compute circuit monotone width
        # This is a dummy implementation and should be replaced with actual computation
        return len(cnf)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        cnf = generate_random_cnf(n, random.randint(2, n))
        state = truth_table_to_quantum_state(cnf)
        mge = calculate_minimal_geometric_entanglement(state, n)
        w = calculate_circuit_monotone_width(cnf)
        results.append((mge, w))

    correlation_coefficient = compute_correlation(results)
    conjecture_holds = correlation_coefficient > 0.8 and all(mge <= 2 * w for mge, w in results)
```

```
    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

def generate_random_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = random.sample(range(-n, 0), random.randint(1, n))
        cnf.append(clause)
    return cnf

def compute_correlation(results):
    if len(results) < 2:
        return 0.0
    x_mean = sum(x for x, _ in results) / len(results)
    y_mean = sum(y for _, y in results) / len(results)
    numerator = sum((x - x_mean) * (y - y_mean) for x, y in results)
    denominator = math.sqrt(sum((x - x_mean) ** 2 for x, _ in results)) * math.sqrt(sum((y - y_mea
    return numerator / denominator if denominator != 0 else 0.0

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractio
    elif any(not r["conjecture_holds"] for r in results) and any(mge > 2 * w for mge, w in (r["met
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={seeds[ne
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_03c93280.py", line 83, in <module>
```

```
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_03c93280.py", line 44, in run_trial
    state = truth_table_to_quantum_state(cnf)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_03c93280.py", line 25, in truth_table_t
   if all(any(assignment[abs(lit) - 1] == l > 0 for l in clause) or any(assignment[abs(lit) -
1] != l < 0 for l in clause) for clause in cnf):
     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_03c93280.py", line 25, in <genexpr>
   if all(any(assignment[abs(lit) - 1] == l > 0 for l in clause) or any(assignment[abs(lit) -
1] != l < 0 for l in clause) for clause in cnf):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_03c93280.py", line 25, in <genexpr>
   if all(any(assignment[abs(lit) - 1] == l > 0 for l in clause) or any(assignment[abs(lit) -
1] != l < 0 for l in clause) for clause in cnf):
                              ^^^
NameError: name 'lit' is not defined. Did you mean: 'List'?
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13542
2	propose	ollama_remote	glm4:latest	0	0	12985
3	preregistration	ollama_remote	glm4:latest	0	0	9416
4	novelty	ollama_remote	glm4:latest	0	0	9704
5	novelty	ollama_remote	glm4:latest	0	0	8403
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27549
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12614
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13617
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11767
10	judge	ollama_remote	glm4:latest	0	0	12615

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 132213 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/4416fd13c45a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4416fd13c45a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4416fd13c45a.tar.gz` (if generated)